

H0009 — Ciclo de vida del torneo

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

Épica: Gestión del torneo · Puntos: 8 · Prioridad KANO: Básico · Rol: Coordinador

Rama: Dev · Fecha: 2026-08-26 · Migración: CategoríaEstado (escrita a mano, con backfill)

1. El problema

El sistema modelaba **estados pero no transiciones**. `EstadoTorneo` existía desde el starter, pero *ningún endpoint lo cambiaba*: todos los torneos quedaban en `Borrador` para siempre. La consecuencia era que los guards de "torneo Finalizado" —escritos y testeados en el bloque CRUD— eran **código correcto que nunca se ejecutaba**.

Sin estados el sistema tampoco podía responder la pregunta que separa dos mundos con reglas opuestas: *¿este torneo se está planificando o se está compitiendo?* Editar una categoría con llaves generadas es deseable mientras se planifica y es corrupción de datos mientras se compete.

Borrador → Activo → Finalizado

(y Activo → Borrador solo si no se compitió)

Momento	Estado	Qué habilita	Qué bloquea
Planificación	Borrador	Crear/editar/eliminar categorías y competidores, generar llaves	Registrar ganadores
Competencia	Activo	Registrar ganadores, avanzar rondas	Tocar categorías, competidores o llaves
Cierre	Finalizado	Consultar y (con H0008) generar reportes	Toda escritura

2. Decisiones de diseño

D1 — `LlavesGeneradas` sobrevive, pero como propiedad derivada

La historia pedía reemplazar el booleano `Categoría.LlavesGeneradas` por el enum `EstadoCategoría`. Reemplazarlo a secas habría tocado guards, clasificador, respuesta de la API y cinco lugares del frontend.

En lugar de eso, el bool quedó **derivado del estado**:

```
public EstadoCategoria Estado { get; set; } = EstadoCategoria.SinLlaves;

public bool LlavesGeneradas => Estado >= EstadoCategoria.LlavesGeneradas;
```

El resultado fue medible: al compilar, **un solo error** en todo el código de producción (la única asignación real, en `GenerarLlavesCommandHandler`). Los guards de edición y borrado, el clasificador y todo el frontend siguieron funcionando sin tocarse.

No es un parche de compatibilidad: la pregunta "*¿esta categoría ya tiene bracket?*" se sigue haciendo en varios lugares y merece un nombre. Lo que cambia es que ahora tiene una sola fuente de verdad en vez de ser un campo que hay que acordarse de mantener.

Por eso el enum tiene valores numéricos explícitos y un comentario que advierte que **el orden es significativo**: se compara con `>=`. Reordenarlo rompería silenciosamente los guards.

D2 — El estado de la categoría se deriva de sus matches

`EstadoCategoria` no se maneja a mano en ningún lado salvo al generar llaves. Al registrar un ganador, el handler lo recalcula:

```
var estadoNuevo = todas.All(1 => 1.Estado is EstadoLlave.Finalizado or EstadoLlave.Bye)
    ? EstadoCategoria.Finalizada
    : EstadoCategoria.EnCurso;
```

Se persiste en vez de calcularse al vuelo para no tener que recorrer todo el bracket cada vez que alguien pregunta si una categoría ya tiene campeón — que es justo lo que necesita la validación de cierre del torneo, y lo que va a necesitar el reporte de H0008.

D3 — Tres transiciones y ninguna más

El handler usa un `switch` sobre la tupla `(estadoActual, destino)`. Todo lo que no esté en esas tres ramas cae en el `default` y devuelve **409**. No se puede saltar de Borrador a Finalizado ni reabrir un torneo cerrado.

Transición	Precondición	Por qué
Borrador → Activo	≥1 categoría con llaves generadas	Activar sin brackets dejaría el torneo trabado : en Activo ya no se pueden generar llaves
Activo → Finalizado	Todas las categorías con llaves, Finalizadas... o forzar	Una categoría puede quedar sin disputarse; es una decisión del Coordinador, no un atajo
Activo → Borrador	Ninguna categoría en <code>EnCurso</code> o posterior	Volver habilitaría rehacer llaves y descartaría en silencio los ganadores cargados

El **forzar** es deliberadamente estrecho: **solo** habilita saltar la validación de cierre. No abre ninguna otra transición. Un flag genérico de "ignorar validaciones" habría sido una puerta trasera a todo el modelo.

Y el mensaje de error dice *cuáles* categorías faltan, no solo que faltan: hay un test que verifica que el nombre de la pendiente aparece y el de la terminada no.

D4 — La transición es idempotente

Pedir el estado en el que el torneo ya está no es un error: devuelve el torneo sin escribir nada. Evita que un doble clic o un reintento de red produzcan un 409 confuso.

D5 — Los torneos finalizados dejan de esconderse

Esto apareció al implementar, no estaba en la historia. El repositorio tenía **GetAllActivosAsync**, que filtraba **Estado != Finalizado**. Con las transiciones recién implementadas, eso significaba que **finalizar un torneo lo hacía desaparecer de la aplicación** — justo en la etapa cuyo propósito es consultarlo y sacarle reportes (H0008).

Pasó a **GetAllAsync**, ordenado por fecha descendente. El estado se muestra en cada card; filtrar o no es decisión de la vista, no del repositorio.

D6 — La migración se escribió a mano

La que generó **dotnet ef** era destructiva:

```
DropColumn("llaves_generadas");
AddColumn<string>("estado", defaultValue: "");
```

Habría **perdido qué categorías ya tenían bracket** y dejado un valor (**"**) que ni siquiera parsea contra el enum. La versión escrita a mano agrega la columna con default **SinLlaves**, **migra los datos** y recién entonces borra la vieja:

```
UPDATE categorias SET estado = 'LlavesGeneradas' WHERE llaves_generadas = true;
```

El **Down** hace el camino inverso. No se puede inferir si una categoría estaba **EnCurso** o **Finalizada** sin mirar sus matches, así que todas arrancan en **LlavesGeneradas**: el primer ganador que se registre las recalcula, y en el peor caso se ve una etapa atrasada — nunca datos perdidos.

3. Qué se implementó

Backend

Archivo	Cambio
Domain/Enums/EstadoCategoria.cs	Enum nuevo, con el orden documentado como significativo

Archivo	Cambio
Domain/Entities/Categoria.cs	Estado + LlavesGeneradas derivada
Configurations/CategoriaConfiguration.cs	Columna estado como string; ignora la derivada
Migrations/..._CategoriaEstado.cs	Reescrita: agrega, migra, borra
.../Commands/CambiarEstadoTorneo/	Command, Validator y Handler con las tres transiciones
API/Endpoints/Torneos/CambiarEstadoTorneo*	PUT /api/v1/torneos/{id}/estado , Roles("Coordinador")
GenerarLlavesCommandHandler	Guard: solo en Borrador. Marca Estado = LlavesGeneradas
RegistrarGanadorCommandHandler	Guard: solo en Activo. Deriva el estado de la categoría
ITorneoRepository / TorneoRepository	GetAllActivosAsync → GetAllAsync (D5)
CategoriaResponse	Expone Estado además de LlavesGeneradas
6 handlers de categorías y competidores	Guard endurecido de "no Finalizado" a "solo Borrador"

Frontend

Archivo	Cambio
src/types/index.ts	Tipo EstadoCategoria ; estado en Categoria
src/lib/api.ts	torneos.cambiarEstado(id, estado, forzar)
src/hooks/useTorneos.ts	useCambiarEstadoTorneo
components/torneos/TorneoEstadoActions.tsx	Botones de transición con ConfirmDialog
components/categorias/CategoriaCard.tsx	Etiqueta de la etapa de la categoría
routes/.../categorias.tsx	Panel de estado + oculta las secciones de planificación fuera de Borrador

El hook invalida **tres** queries (listado, detalle y categorías del torneo). El estado cambia qué acciones se habilitan en casi toda la app: dejar cualquiera de esas queries vieja mostraría botones que el backend ya rechaza.

Cada transición pasa por **ConfirmDialog** con una descripción de *qué se habilita y qué se bloquea*, porque en la práctica son irreversibles.

4. Tests

+21 tests (109 → 130). Los 12 nuevos del handler de transición cubren:

- Las tres transiciones válidas, cada una con su precondition cumplida.
- Activar sin ninguna llave generada → 409.
- Finalizar con categorías pendientes → 409 *con sus nombres en el mensaje*.

- Finalizar forzado → ignora las pendientes.
- Volver a Borrador con resultados (EnCurso y Finalizada) → 409.
- Borrador → Finalizado y reabrir un Finalizado → 409 (no se saltean estados).
- Mismo estado → idempotente, sin persistir.

Más 9 en los handlers existentes: los guards de estado de generar y registrar, la derivación de `EnCurso` / `Finalizada`, y los cuatro tests de CRUD que pasaron de `[Fact]` a `[Theory]` para cubrir también **Activo**, no solo Finalizado.

130/130 tests en verde. Backend sin errores C#; frontend compila y pasa lint.

5. Cómo verificarlo

1. Reiniciar la API: aplica `CategoriaEstado` y migra las categorías existentes.
2. En un torneo en Borrador, generar llaves. La card de categoría muestra "✓ *Llaves generadas*".
3. Panel **Estado del torneo** → *Iniciar torneo*. Confirmar.
4. Al volver a Categorías: desaparecen "Armar las llaves" y "Nueva categoría"; los botones de editar y eliminar categoría ya no aparecen.
5. Registrar un ganador → la categoría pasa a "● *En curso*". Al resolver el último match, "✓ *Finalizada — hay campeón*".
6. *Finalizar torneo* con categorías a medias → error nombrando cuáles. *Finalizar de todos modos* lo fuerza.
7. El torneo finalizado **sigue apareciendo** en el listado, con su badge de estado (D5).

6. Notas para las próximas historias

- **H0010 (rehacer llaves)** es la siguiente y ahora tiene dónde apoyarse: el guard natural es "solo con el torneo en Borrador". Sigue pendiente **reescribir la regla 4 de** `skills/bracket-algorithm.md`, que todavía dice "no regenerar" sin matices.
- **H0008 (reporte PDF)** puede usar `EstadoCategoria.Finalizada` para saber qué categorías tienen campeón sin recorrer los brackets.
- **Editar el torneo** (nombre, fecha, lugar) sigue permitido en Activo: solo se bloquea en Finalizado. Es deliberado — corregir el lugar de un torneo en curso es legítimo y no toca datos de competencia.